AULA 15

Navegação autônoma: o Nav2 em cima do seu mapa

Etapa 8 · terça, 27/10/2026 · sala SJ205 · Prof. Dácio Moreira de Souza

A nuvem concentrar prova confiança, não acerto.

O gate desta sexta pede um print do RViz2 com a nuvem de partículas concentrada. Esse print é exatamente o que NÃO distingue um robô localizado de um robô 5,92 m fora do lugar, com o rumo invertido. Hoje a gente mede a diferença.

Cenas dos últimos episódios: onde paramos

- Na semana passada: o enunciado do TP3 lido item por item, e a entrega na sexta
- O que ficou dito ali vale hoje em outra escala — o enunciado é a rubrica
- O mapa que vocês salvaram deixa de ser evidência de entrega e passa a ser ENTRADA
- E a aresta map → odom, que na Aula 11 era mentira, hoje passa a ser calculada

As peças de hoje, todas suas

- Aula 11 — a odometria erra 3% e o erro só cresce; map → odom publicava identidade
- Aula 12 — o SLAM entregou o par .pgm + .yaml; hoje é nele que o robô se acha
- Aula 9 — a régua se declara ANTES de medir, senão ela se ajusta ao resultado
- Aula 13 — achar não é acertar: o maior IoU da tabela era do pior detector
- Aulas 7 e 8 — ações. O Nav2 É um servidor de ação: vocês já sabem navegar

Como a aula funciona hoje

- A ordem é por custo de perda: o que vence antes vem primeiro
- Bloco 1, a nuvem medida, sem ROS 2 — é a ideia da aula e roda em qualquer máquina
- Bloco 2, o Nav2 no ar com o seu mapa — é o que vence nesta sexta, 30/10
- Bloco 3, um goal e depois a rota de cinco pontos, por código
- Bloco 4, o TP4 item por item — e é ele que cede se o tempo apertar

Primeira coisa do dia: instalar o Nav2

```
sudo apt update
sudo apt install -y ros-humble-navigation2 ros-humble-nav2-bringup

# as tres linhas tem de responder:
ros2 pkg list | grep -c '^nav2_'          # 20 ou mais
ros2 pkg executables nav2_amcl            # nav2_amcl amcl
ros2 pkg executables nav2_bt_navigator    # nav2_bt_navigator bt_navigator
```

São 200 MB e dois minutos. Deixe baixando enquanto a gente conversa — o resto do dia depende dele. O nav2_bringup entra porque é dele que vem o display da nuvem no RViz2.

Nav2 vem do apt, na versão do Humble.

Não compile o Nav2 do main e não instale nada dele por pip: os nomes de parâmetro mudaram entre versões, e o sintoma não diz que foi versão — o que aparece é o gerenciador de ciclo de vida parado em "Configuring", calado, para sempre. Terceira vez no semestre: Humble pareia com Fortress, cv_bridge pareia com NumPy 1.

BLOCO 1

A nuvem que concentrou no lugar errado

Medir a convergência sem ROS 2 e sem Nav2

```
# ■■ herda PB_USER, PB_DIR e PB_WS do bloco de preparação ■■
cd "$PB_DIR/aula15-nav2"
python3 simular_loc.py        # os tres cenarios
python3 avaliar_loc.py        # a tabela comparada
```

Um filtro de partículas honesto em NumPy, sobre o mapa da Aula 11 — que aqui serve de mapa conhecido E de verdade. É por isso que ele mede o que o RViz2 não mostra.

A tabela, e ela é determinística

cenario	t conv (s)	erro pos (m)	erro ang (g)	nuvem (m)	N ef min	convergiu para
uniforme	3.0	0.02	0.4	0.066	11.8	lugar certo
aproximada	0.6	0.02	0.5	0.063	63.0	lugar certo
errada	0.6	5.92	179.7	0.078	114.5	lugar ERRADO

Semente 42: rodando de novo sai idêntico, na sua máquina e na minha. Leia a coluna da nuvem e a do erro JUNTAS — é a aula inteira em duas colunas.

Os três terminam com a nuvem do mesmo tamanho.

6,3 cm, 6,6 cm e 7,8 cm de raio. Nos três, o print do RViz2 mostra a mesma manchinha concentrada, status Ok, nenhum aviso. E um deles está a 5,92 m da verdade, com 180° de erro de rumo — e é o que tem a nuvem MAIS concentrada.

Quatro leituras da tabela

- O erro não dá erro: a pose inicial errada não levanta exceção nem aviso
- Ela converge MAIS RÁPIDO que as outras duas, e com a nuvem mais apertada
- Olhe o N ef min: no uniforme despenca a 11,8 de 1200 — o filtro passou por uma crise
- No errada ele nunca desce de 114,5: o filtro nunca teve dúvida. E é coerente
- A localização ganha da odometria por 30× — 0,02 m contra 0,72 m no fim do laço

Concordância interna não é evidência.

Todas as partículas concordam porque todas estão igualmente erradas. É a Aula 13 de novo: qualquer métrica que mede o que o algoritmo acha de si mesmo se engana da mesma maneira. A cura também é a mesma — medir contra a verdade.

A sala gêmea: por que o gate pede o clique

```
cd "$PB_DIR/aula15-nav2"
python3 avaliar_loc.py uniforme --particulas 400 --segundos 20

uniforme           8.4         8.48        178.1      0.223      13.5  lugar ERRADO
```

O mundo é quase simétrico por rotação de 180°, e ninguém projetou isso. Com pouca partícula e pouco percurso, a nuvem trava na sala gêmea: 8,48 m fora. Não é bug — é o que simetria faz com um filtro sem informação a priori. O SEU mapa também tem uma: corredores iguais, salas espelhadas, um hall vazio. E a resposta não é mais partículas, é dar a pose inicial — que é por isso que set_initial_pose existe no AMCL.

BLOCO 2

O Nav2 no ar, e o gesto que decide

O launch, e o --show-args antes dele

```
# ■■ herda PB_USER, PB_DIR e PB_WS do bloco de preparação ■■
ros2 launch aula15_nav2 nav2.launch.py --show-args     # SEMPRE antes
ros2 launch aula15_nav2 nav2.launch.py rviz:=true
```

ros2 launch aceita argumento desconhecido CALADO: rviz:=true num pacote que não declara rviz não faz nada. Se a lista não trouxer mapa, rviz, pose_inicial_auto e deriva, o pacote compilado é outro — quase sempre o cp -r aninhado.

As nove peças, e por que sobem em duas levas

- O mundo da Aula 11, com map_odom:=false e piloto:=false
- map_server lê o .yaml e publica /map; amcl publica map → odom
- O gerenciador de localização ativa esses dois, nesta ordem
- Depois: controller (DWB), planner (NavFn), smoother, behavior, bt_navigator...
- ...waypoint_follower e velocity_smoother — e o segundo gerenciador ativa os sete

Nav2 é software de ciclo de vida.

Nó no ar não é nó ativo: cada um precisa ser configurado e ativado, e quem faz isso é o gerenciador. Se um nome da lista dele não corresponder a um nó realmente no ar, ele espera para sempre — sem erro, sem aviso, parado em Configuring.

Os três comandos que separam os casos

```
ros2 lifecycle get /amcl              # active [3]
ros2 lifecycle get /bt_navigator      # active [3]
ros2 node list                        # quem esta no ar
```

Decore estes três. "O Nav2 não navega" quase sempre é um deles respondendo outra coisa — e o jeito número um de perder uma hora é procurar no lugar errado.

A cadeia do /cmd_vel, que é onde todo mundo tropeça

launch/nav2.launch.py

```
remap_controlador = [("cmd_vel", "cmd_vel_nav")]
remap_suavizador  = [("cmd_vel", "cmd_vel_nav"),
                     ("cmd_vel_smoothed", "cmd_vel")]

# controller_server -> /cmd_vel_nav -> velocity_smoother -> /cmd_vel -> mundo
```

São os mesmos remapeamentos do navigation_launch.py oficial. Tire o velocity_smoother sem desfazê-los e o controlador fala em /cmd_vel_nav, ninguém ouve: o robô não anda e nenhum nó reclama. Confira com ros2 topic info /cmd_vel --verbose: UM publicador.

O 2D Pose Estimate: o gesto mais importante do dia

- O launch sobe com a pose inicial MANUAL, de propósito. O AMCL espera
- Clique na posição, arraste na direção do rumo, e OLHE a nuvem nascer onde você clicou
- Faça isso com a mão pelo menos uma vez: você vê a nuvem fechar — ou não
- Para repetir um teste sem reclicar: pose_inicial_auto:=true
- Sem pose inicial não há map → odom, e o bt_navigator fica esperando a TF

A convergência pelo lado dos números

```
ros2 topic echo /amcl_pose --once      # a pose E a covariancia
ros2 run tf2_ros tf2_echo map odom     # a correcao que o AMCL aplica
```

A covariância é a sua nuvem em número. E o tf2_echo map odom é o erro acumulado da odometria com o sinal trocado: com a deriva de 3% da Aula 11, essa translação cresce ao longo do laço. É literalmente a conta que o AMCL está pagando.

BLOCO 3

Um goal, e depois a rota

O Nav2 é um servidor de ação.

navigate_to_pose tem objetivo, feedback, resultado e cancelamento — os quatro motivos pelos quais ação existe, e o conteúdo das Aulas 7 e 8. Quem entendeu aquelas duas aulas já sabe navegar: o que faltava era o nome do servidor.

Do clique no RViz2 para o código

```
ros2 action list | grep navigate           # navigate_to_pose esta ai
ros2 run aula15_nav2 rota                  # a rota padrao, uma volta
ros2 run aula15_nav2 rota --ros-args -p voltas:=0     # patrulha sem fim
```

Primeiro mande um goal pelo botão Nav2 Goal e veja o plano global aparecer. Depois o objetivo vira código — e o rota.py tem 150 linhas e é o item 1 do TP4 inteiro.

As três linhas do rota.py que valem ler

aula15_nav2/rota.py

```
alvo.header.frame_id = quadro          # 'map', nao 'base_link'

if status == GoalStatus.STATUS_SUCCEEDED:   # no Humble o result e
    ...                                     # um std_msgs/Empty

rclpy.spin_until_future_complete(self, envio)   # no main, NUNCA
                                                # dentro de callback
```

Goal em base_link pede "ande 2 m para a frente": o Nav2 obedece, sem reclamar. error_code no result é de versão posterior ao Humble. E spin dentro de callback travaria o nó para sempre — nada acontece e nada reclama. É o cuidado da Aula 8.

O desvio reativo que o item 1 pede

- No Nav2 não é nó novo: é o costmap LOCAL alimentado pelo /scan, e o DWB decidindo
- O global é o mapa inteiro com inflação — é sobre ele que o NavFn planeja
- O local é uma janela rolante de poucos metros, reconstruída do laser a cada ciclo
- Explicar essa divisão com os números do SEU YAML entrega o item bem feito

A porta tem 1,2 m e o robô tem 0,40 m.

Com inflation_radius 0,35 sobram 0,50 m de corredor livre: passa, com 5 cm de folga de cada lado. Aumente para 0,50 e a porta FECHA — o planejador devolve "No valid path could be found" e o aluno conclui que o Nav2 não funciona. Faça essa conta com as passagens do seu mapa antes de culpar o planejador.

BLOCO 4

O TP4, item por item

Quatro itens, e o verbo que pega em cada um

- 1. Nav2 + nó de rota — ENVIANDO goals sequenciais, com desvio reativo
- 2. Árvore de comportamento + bag — INTERROMPA, AGUARDE, RETOME, e REPRODUZÍVEL
- 3. Percepção veicular no MetaDrive — SOBREPONHA ao frame da câmera
- 4. CNN com 500 frames em 3 classes — COMPARE as curvas com e sem augmentation

Item 1 — onde ele quebra

- O enunciado diz Gazebo: a versão da disciplina é Fortress, e o mundo mínimo é aceito
- Diga isso no relatório em uma frase. Não troque de simulador a três semanas da entrega
- robot_radius e inflation_radius mudam com o SEU URDF — e decidem se a passagem abre
- Metade dos "o Nav2 não navega" é AMCL sem pose inicial
- Ponto em célula ocupada é goal RECUSADO, não goal que falhou: o servidor nem tenta

Item 2 — a árvore, e onde mora a decisão

- Os XML prontos estão em /opt/ros/humble/share/nav2_bt_navigator/behavior_trees/
- Copie um para o SEU pacote; o campo behavior_tree do goal aponta para ele
- Caminho completo: nó de condição próprio em C++, com Wait de 2 s num ReactiveFallback
- Caminho de cliente: a árvore padrão, e o seu nó cancela, espera 2 s e reenvia
- Os dois passam. O que não passa é não dizer qual você usou — traga a sua escolha em 03/11

Item 2 — a gravação, e o campo que todo mundo esquece

```
mkdir -p "$PB_DIR/bags" && cd "$PB_DIR/bags"
ros2 bag record -o sessao-tp4 \
  /amcl_pose /odom /scan /cmd_vel /plan /tf /tf_static

# depois, com TUDO desligado:
ros2 bag info sessao-tp4 && ros2 bag play sessao-tp4
```

O /tf_static é publicado UMA vez, no instante em que o sistema sobe. Sem ele no record, a reprodução é um laser flutuando sem corpo — e você só descobre depois, quando não há como regravar. Plote /odom e /amcl_pose no mesmo eixo: é a Aula 11 numa figura.

Item 3 — MetaDrive: o que eu sei e o que eu não sei

- A 0.4.3 é a última publicada (dez/2024) e aceita o Python 3.10 do Ubuntu 22.04
- Instala com uv: uv venv --system-site-packages, uv pip install --python <venv>/bin/python
- A documentação afirma suportar renderização sem tela — e traz o verify_headless
- EU NÃO RODEI o MetaDrive. Quem rodar primeiro, diga em sala: vale para a turma
- Salve os FRAMES em disco e trabalhe nos arquivos: o resto do TP não depende dele subir
- Se não subir, o cena_estrada.py da Aula 13 é saída declarada — e já mede as falhas

Item 4 — a CNN, e a parte que não comprime

- O que decide o item não é a rede: é a COMPARAÇÃO com e sem augmentation
- Mesma arquitetura, mesma semente, mesmas épocas — senão você mede sorte, não augmentation
- Os 500 frames rotulados têm de ser vistos um a um. Comece nesta semana
- 400 de via livre e a rede acerta 80% prevendo sempre via livre: por isso a matriz
- Espelhar horizontal troca o lado da pista — pense antes de ligar, e escreva o que pensou
- TensorFlow é script solto, nunca dentro de nó ROS 2: é o _ARRAY_API not found de volta

A tarefa da semana

- O Nav2 no SEU projeto, com o SEU mapa, o AMCL convergindo e uma rota de três pontos
- Molde: launch, nav2.yaml e nav2.rviz do exemplo — mexa nas cinco linhas marcadas # TROQUE
- Cinco linhas de verificação: lifecycle amcl, lifecycle bt_navigator, topic info /cmd_vel...
- ...tf2_echo map odom (NÃO identidade) e ros2 action list | grep navigate
- E a evidência commitada: o print da nuvem MAIS a frase que diz como você sabe que é ali

Desafio — medir a ambiguidade do seu mapa

- Varra as partículas: 200, 400, 800, 1200, 2000. Em que N ele passa a achar o lugar certo?
- E muda com a semente? Quanto?
- Ou, no seu Nav2 real: dê de propósito a pose inicial no lugar simétrico do seu mapa
- A nuvem fecha? Em quantos segundos? Com que covariância?
- Se as duas covariâncias forem parecidas, você reproduziu a tabela de hoje no seu projeto

Em 03/11: a árvore e a sessão gravada.

O XML, onde mora a decisão de interromper, e o bag que reproduz com o robô desligado. Chegue com o Nav2 navegando no seu projeto e com a sua escolha de caminho para a interrupção — a aula é sobre a sua escolha, não sobre a minha. E comece a salvar os frames do MetaDrive nesta semana.